A Small Work-Stealing Task Runtime

Roger Yang (hryang)

15-418 Project Proposal

URL: `https://rogeryang85.github.io/work-stealing-runtime/`

Summary

I'm going to write a small Cilk-like task runtime in C++ with spawn/sync and per-thread work-stealing deques. I'll benchmark it on the GHC machines against a simple shared-queue version and against OpenMP tasks.

Background

Lots of parallel code is recursive: you split the problem, spawn a task for one half, do the other half yourself, then wait. For example:

```
int fib(int n) {
  if (n < 2) return n;
  int a, b;
  spawn([&]{ a = fib(n - 1); });
  b = fib(n - 2);
  sync();
  return a + b;
}
```

This creates a huge number of tiny tasks, and the runtime has to spread them over a few threads. In work stealing, each thread keeps its own deque of tasks. It pushes and pops from the bottom, and when it runs out of work it steals from the top of someone else's deque. Most of the time threads only touch their own deque, so there's not much synchronization. I'll use the Chase–Lev lock-free deque [2] and do child stealing (a thief takes the spawned task, like OpenMP and TBB do). A thread waiting at a sync keeps running other tasks instead of blocking.

The Challenge

- **The deque.** The owner and a thief can race for the last task, and it's easy to get the memory orderings wrong. x86 hides a lot of these bugs, so I'll also test on an ARM Mac.
- **Overhead.** Tasks in fib do almost nothing, so things like `malloc` per task or an atomic counter on every sync can end up costing more than the actual work. I need the common case (spawn a task, pop it right back) to be cheap, and I need to figure out when to stop spawning.

- **Workloads.** fib and n-queens are compute-bound and fairly balanced. Quicksort depends on pivots and cares about cache locality. Unbalanced Tree Search (UTS) [4] is very irregular, so lots of steals happen and contention on victims becomes the bottleneck. A central queue should do badly here.

Resources

GHC machines (i7-9700, 8 cores) for timing and an M-series Mac for testing on a weaker memory model. I'm starting from scratch with C++17 `std::thread`/`std::atomic`. Baselines are OpenMP tasks with GCC (libgomp) and Clang (libomp). Main references are the Cilk-5 paper [1], Chase–Lev [2], and Lê et al. [3], which gives a version of the deque that's correct on weak memory models. I may ask for PSC access if I get to the scaling stretch goal.

Goals and Deliverables

Plan to achieve:

- Working runtime: spawn/sync, Chase–Lev deque per thread, random stealing.
- A central shared-queue runtime with the same API, for comparison.
- fib, quicksort, n-queens and UTS, each in serial, my runtime, the central queue, and OpenMP.
- Per-thread task pools instead of `malloc`, and at least two other steal policies (round-robin, last successful victim).
- Stats per thread (steals, failed steals, idle time) and speedup graphs for 1–8 threads.
- Targets: around 6× or better on n-queens and UTS with 8 threads, beat libgomp on the fine-grained benchmarks (it's known to have high per-task overhead), and get close to libomp.

Hope to achieve: match or beat libomp on UTS; adaptive granularity (lazy binary splitting [5]) so I don't need hand-picked cutoffs; spin vs. backoff vs. sleep for idle threads; scaling runs on a bigger PSC node.

If things go slowly: a correct work-stealing runtime with random stealing, compared against the central queue and OpenMP on all four benchmarks.

Poster: speedup graphs, plus a per-thread timeline for UTS showing when each thread was working, stealing, or idle.

Questions I want to answer: Where does the per-task time go? At what task size does the central queue stop scaling? Which steal policy works best for balanced vs. irregular workloads, and why?

Platform Choice

Work stealing is built for shared-memory multicore CPUs, and recursive, irregular tasks don't fit GPUs or SIMD well. C++ gives direct control over memory ordering and cache-line layout directly, which matters for the deque.

Schedule

Week	Tasks
Nov 9–15	Thread pool, deque, stress tests on ARM and x86, spawn/sync, fib working.
Nov 16–22	Quicksort, n-queens, UTS; serial and OpenMP versions; central queue; first speedup graphs.
Nov 23	**Milestone due**
Nov 23–29	Thanksgiving, so lighter. Task pools, per-thread stats, more steal policies.
Nov 30–Dec 6	Policy and granularity experiments; stretch goals if I'm ahead.
Dec 7–10	Final runs, timeline figure, report. **Report due Dec 10.**
Dec 11	**Poster session**

References

[1] Frigo, Leiserson, Randall. The implementation of the Cilk-5 multithreaded language. PLDI 1998.

[2] Chase, Lev. Dynamic circular work-stealing deque. SPAA 2005.

[3] Lê, Pop, Cohen, Zappa Nardelli. Correct and efficient work-stealing for weak memory models. PPoPP 2013.

[4] Olivier et al. UTS: An unbalanced tree search benchmark. LCPC 2006.

[5] Tzannes, Caragea, Barua, Vishkin. Lazy binary-splitting: a run-time adaptive work-stealing scheduler. PPoPP 2010.